

EE 475 Introduction to Image Processing

Homework 05

Boran Aybak Kilic
Student ID: 2022401207

November 13, 2025

Disclaimer: The report length guideline of a maximum of two single-column pages was acknowledged but deemed insufficient given the scope and extent of the required implementations and analysis. Permission was granted by Professor Alpay Ozcan to submit a longer, more detailed report. While the primary objective is to satisfy evaluation criteria (which one also can do so by looking at the results and skimming through the methods), the comprehensive documentation of methods and results is essential for maximizing learning outcomes and personal technical development.

1. Part (a): Custom Convolution Function

Per the assignment, a custom 2D convolution function, `my_convolution.m` (see Appendix 1), was implemented without using built-in libraries. The convolution operation $G = I * K$ computes an output pixel $G(i, j)$ by taking the weighted sum of the image neighborhood centered at (i, j) with the *flipped* kernel. This is mathematically defined as:

$$G(i, j) = \sum_m \sum_n I(i - m, j - n) K(m, n) \quad (1)$$

where $I(i - m, j - n)$ is the image neighbourhood and $K(m, n)$ is the kernel. This is equivalent to performing a 2D correlation (an element-wise multiplication and sum) between the image neighborhood and a 180-degree flipped version of the kernel. This flip operation was implemented in the code using `flip(flip(kernel,1) 2)`.

To handle border effects and ensure the output image has the same dimensions as the input ('same' convolution), the input image was first padded with zeros. For a kernel of size $K_h \times K_w$, the required padding on the top and bottom boundaries is $p_h = \lfloor K_h/2 \rfloor$, and on the left and right boundaries is $p_w = \lfloor K_w/2 \rfloor$. This was implemented using the `padarray` function.

It is generally desirable to choose an odd-sized kernel (e.g., 3x3, 5x5), as this provides an unambiguous center pixel, making the convolution operation symmetric around the pixel (i, j) .

To validate the function, its output was compared against MATLAB's built-in `conv2` function. While the built-in function may utilize a slightly different approach for treating boundaries, especially with even-sized kernels, the results for an odd-sized kernel are identical. As specified, a Gaussian kernel with an odd size (size=11, sigma=7) was used for the test. The results, shown in Figure 5 in the Appendix, are exactly the same, which confirms the correctness of the custom implementation.

2. Part (b): Gradient Image (2-Norm)

A function, `my_gradient_norm.m` (see Appendix 2), was implemented to compute the 2-norm (magnitude) of the image gradient. This function uses the custom `my_convolution` function

from Part (a) to convolve the image with the Sobel kernels for the horizontal (K_x) and vertical (K_y) derivatives.

The Sobel kernels are defined as:

$$\mathbf{K}_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad \mathbf{K}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix} = \mathbf{K}_x^T$$

As discussed in the textbook (Digital Image Processing by Gonzalez, pp. 185-186), the derivatives G_x and G_y are implemented using convolution:

$$G_x(x, y) = I(x, y) * K_x \quad \text{and} \quad G_y = I(x, y) * K_y$$

It is important to note that our `my_convolution` function performs a true mathematical convolution, meaning it flips the kernel 180 degrees before correlating. Since the Sobel kernels are already defined in their correlation-ready form, convolving with them ($I * K_x$) technically computes the *negative* of the derivative.

Many popular numerical libraries (like NumPy or PyTorch) often implement "convolution" as a correlation (without the kernel flip) for efficiency. However, in our case, this sign difference is irrelevant. We are computing the gradient 2-norm (magnitude), $M(x, y)$, which squares both components:

$$M(x, y) = \sqrt{G_x^2(x, y) + G_y^2(x, y)} = \sqrt{(-G_x(x, y))^2 + (-G_y(x, y))^2}$$

Therefore, the sign flip is canceled, and the resulting magnitude is correct. A demonstration of this function on a standard test image is provided in Figure 6 in the Appendix.

3. Part (c): Edge Detection and Comparison

This part required running two different edge detection routines on the contact lens image. The first routine is the Gradient 2-Norm, $M(x, y)$, as implemented in Part (b). The second routine is a direct convolution with a kernel designed to detect edges.

The kernel chosen for this second routine was the Laplacian kernel. The Laplacian is a 2nd derivative operator, $\nabla^2 I$, which is highly sensitive to the "curvature" of the intensity profile. This makes it an excellent tool for finding edges, as it produces a strong response (a zero-crossing) at the exact center of an edge or a peak response at the center of a line.

The specific kernel used was:

$$\mathbf{K}_{\text{Lap}} = \begin{bmatrix} 1 & 1 & 1 \\ 1 & -8 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

The intuition behind this kernel, particularly the -8 center, is that it calculates the difference between a center pixel and its 8 neighbors. For the contact lens, the edge is a thin, dark line on a brighter background (a "trough"). When the kernel is centered on this dark line, the center pixel has a low value, while its 8 neighbors have high values. The convolution result is $(\sum \text{neighbors}) - 8 \times (\text{center})$, which yields a large positive value, identifying the dark line. Since Laplacian kernel has kind of a circular symmetry, compared to Sobel kernels, it can be used for general purpose edge detection independent of the orientation of the edge (See Fig. 1).

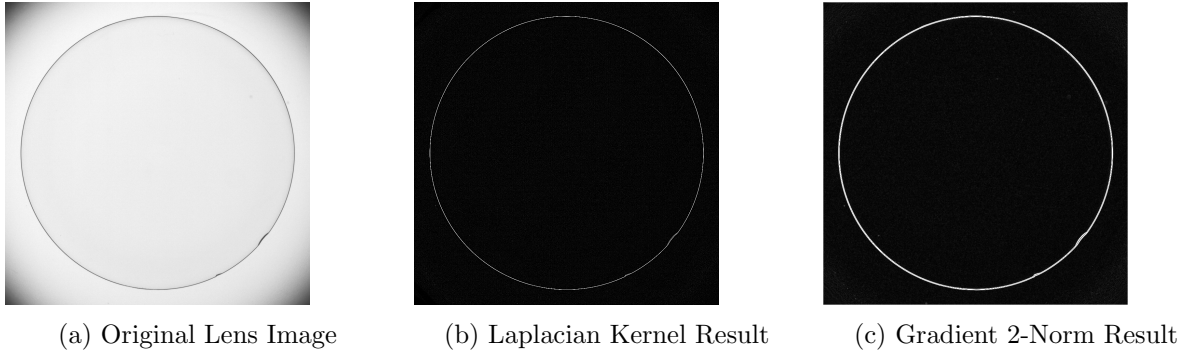


Figure 1: Comparison of edge detection outputs on the contact lens image. Size of the figure and how latex engine compiles the images doesn't do justice to details in the images. See 8, 7 for the scaled up figures.

The two methods produce visibly different results.

- **Gradient 2-Norm (Fig. 1c):** This 1st derivative method produces a thick, "double line" because it detects both slopes (inner and outer) of the dark trough. This is robust and less prone to breaks.
- **Laplacian Kernel (Fig. 1b):** This 2nd derivative method produces a very thin, "single line" by pinpointing the center of the dark trough. While this provides a precise mask for fitting, its thinness is a potential drawback. As a 2nd derivative operator, it is highly sensitive to noise, and the resulting line can be broken or incomplete. This could cause the algorithm to miss defective regions if the edge in that area is not sharp enough to produce a strong response.

Neither method is universally "better"; the gradient is more robust, while the Laplacian is more precise. For the subsequent steps, the Laplacian was chosen, but its limitations were noted.

4. Part (d): Defect Identification

The final part of the assignment required 3 main steps: mask creation, center estimation, and defect classification. As per the instructions, this analysis was performed on *both* edge maps generated in Part : the Gradient 2-Norm (`edge_grad`) and the Laplacian (`edge_lap`).

The process The full MATLAB script for this analysis is available in Appendix D (a single script was used, toggling the `edge_map` variable).

Step 1: Mask Creation and Artifact Handling

A critical problem with both `edge_grad` and `edge_lap` is the presence of high-intensity "edges" at the image border and in the corners. This is caused by two factors:

1. **Convolution Artifacts:** The `my_convolution` function uses zero-padding. When the kernel (e.g., Sobel) reaches the image boundary, it sees a sharp, artificial "cliff" where the image data drops to zero, creating a strong false edge.
2. **Vignetting:** The original image exhibits natural vignetting (darkening) in the corners. The edge detection kernels correctly identify this as a large-area intensity change.

To solve this, the analysis code (3) first crops the edge map (laplacian or gradient norm image obtained in previous steps) to exclude a 2-pixel border: `edge_map(3:end-2, 3:end-2)`. This completely removes the false convolution artifacts. The threshold is then applied to this cropped, "clean" map. To restore the correct coordinates for plotting, the mask is padded back to its original size: `edge_mask = padarray(cropped_edge_mask, [2, 2], ...)`.

Step 2.1: Initial Center Estimation (Centroid)

As suggested in the homework prompt, a good initial guess for the center (C_x, C_y) can be found using a simple "center of mass" or centroid calculation. This method takes the mean of all N pixel coordinates (x_i, y_i) in the binary edge mask:

$$C_x = \frac{1}{N} \sum_{i=1}^N x_i \quad \text{and} \quad C_y = \frac{1}{N} \sum_{i=1}^N y_i$$

This was implemented using `center_x_guess = mean(cols)` and `center_y_guess = mean(rows)`. While fast, this guess is highly sensitive to noise and data outliers (like the corner artifacts), making a more robust method necessary.

Step 2.2: Robust Center Fit (Least-Squares)

To improve the centroid guess, a least-squares circle fit was implemented. The standard non-linear circle equation, $(x - a)^2 + (y - b)^2 = r^2$, is rearranged into a form that is *linear* in its parameters:

$$x^2 + y^2 + c_1x + c_2y + c_3 = 0$$

We can find the relationship between the parameters (c_1, c_2, c_3) and the geometric center (a, b) and radius r by expanding the original equation:

$$\begin{aligned} (x^2 - 2ax + a^2) + (y^2 - 2by + b^2) &= r^2 \\ x^2 + y^2 + (-2a)x + (-2b)y + (a^2 + b^2 - r^2) &= 0 \end{aligned}$$

By equating the coefficients of the two linear forms, we find the mapping:

- $c_1 = -2a \implies a = -c_1/2$
- $c_2 = -2b \implies b = -c_2/2$
- $c_3 = a^2 + b^2 - r^2 \implies r = \sqrt{a^2 + b^2 - c_3}$

This transforms the problem into finding the vector $\mathbf{c} = [c_1, c_2, c_3]^T$ that best fits all N edge points. We can write this as an overdetermined linear system $\mathbf{A}\mathbf{c} = \mathbf{B}$:

$$c_1x_i + c_2y_i + c_3 \cdot 1 = -(x_i^2 + y_i^2) \quad \text{for } i = 1 \dots N$$

This is represented in matrix form and solved in MATLAB as:

$$\mathbf{A} = \begin{bmatrix} x_1 & y_1 & 1 \\ \vdots & \vdots & \vdots \\ x_N & y_N & 1 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} -(x_1^2 + y_1^2) \\ \vdots \\ -(x_N^2 + y_N^2) \end{bmatrix}, \quad \mathbf{c} = \mathbf{A} \setminus \mathbf{B}$$

Step 3: Defect Classification

With a robust center (a, b) and radius r found, the final step was to identify defects. For each point (x_i, y_i) on the clean mask (pixels determined to be on the boundary of the lens), its actual polar radius ρ_i was calculated:

$$\rho_i = \sqrt{(x_i - a)^2 + (y_i - b)^2}$$

A point was classified as defective if its distance from the ideal circle exceeded a tolerance ($2\epsilon = 6$ pixels, chosen to be robust to the boundary thickness): $|\rho_i - r| > \epsilon$.

Final Results

The full analysis pipeline was run on both edge maps.

Routine 1 (Gradient 2-Norm): The `edge_grad` map produced a thick, double-lined mask. After cropping and cleaning, the least-squares fit was successful. As shown in Figure 2, the robust fit finds a good center, and the main defects are correctly identified.

Routine 2 (Laplacian): When the analysis was first run on the 3x3 `edge_lap` from Part (c), a problem arose. The 3x3 kernel is highly sensitive to single-pixel noise, causing it to detect many local minima/maxima that were not part of the true lens boundary as shown in 3. This created a noisy `edge_mask` that corrupted the circle fit.

To solve this, a 5x5 Laplacian kernel was implemented. **The intuition is that a larger kernel is more robust to noise.** By integrating information over a larger 25-pixel neighborhood (vs. 9 pixels), the 5x5 kernel effectively averages out isolated noise points and responds only to the larger, persistent curvature of the actual lens boundary. This 5x5 Laplacian produced a much cleaner mask, which was then used for the final analysis (Figure 4).

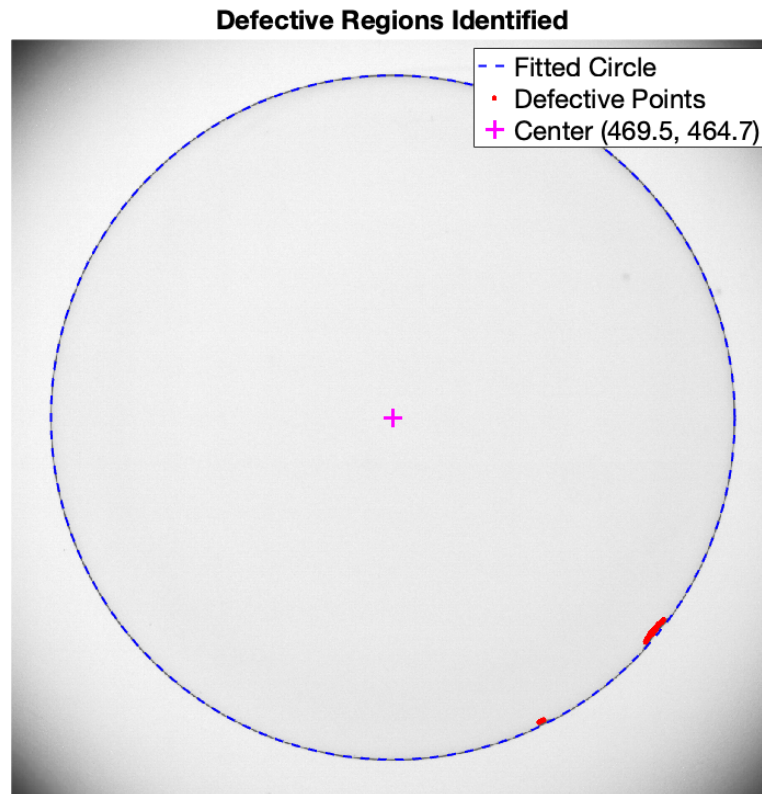


Figure 2: Final result using the **Gradient 2-Norm** map. The fit is robust and correctly identifies the primary defect.

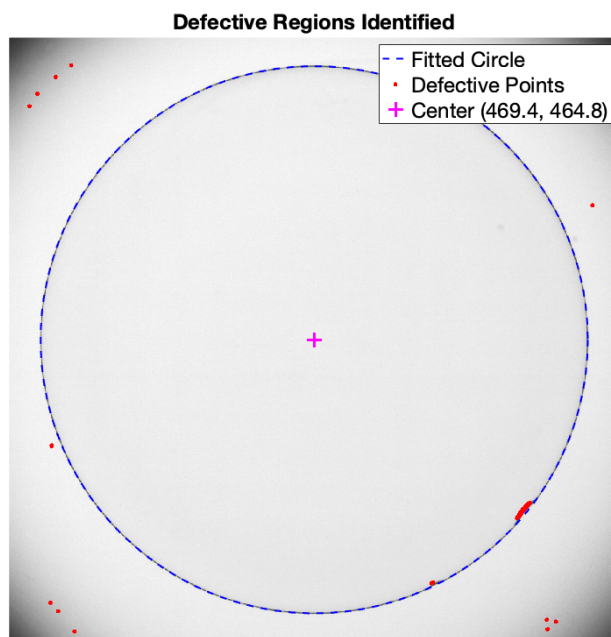


Figure 3: Final result using the **3x3 Laplacian** map. This mask is susceptible to noise and edge mask picks up some pixels that are not on the boundary of the lens.

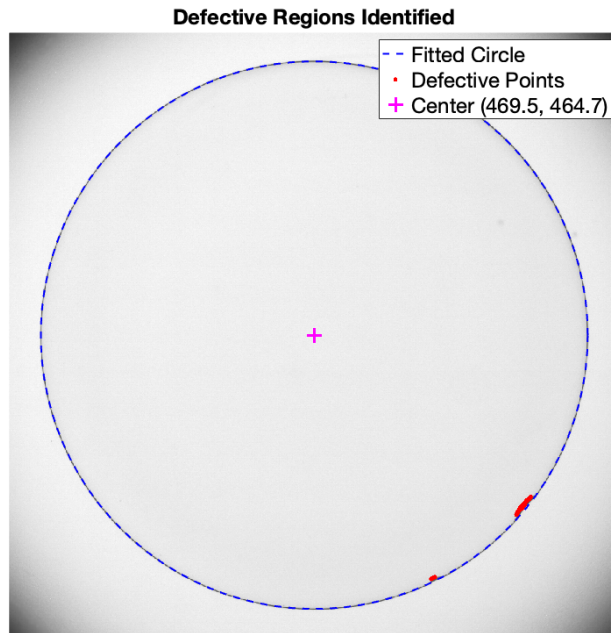


Figure 4: Final result using the **5x5 Laplacian** map. This mask was cleaner, leading to a precise fit that also identifies the defect.

A Code Listings

```
1 function output = my_convolution(image, kernel)
2
3     % Get dimensions of image and kernel
4     [img_rows, img_cols] = size(image);
5     [kern_rows, kern_cols] = size(kernel);
6
7     % Determine kernel "radius" for padding
8     % If (for example) kernel size is 3x3, we will pad the image 1 pixel from
9     both top and bottom
10    half_k_row = floor(kern_rows / 2);
11    half_k_col = floor(kern_cols / 2);
12
13    % Flip the kernel 180 degrees or simply invert the order of array in both
14    dimension
15    flipped_kernel = flip(flip(kernel,1),2);
16
17    % Pad the input image with zeros to handle borders
18    % This ensures the output image is the same size as the input
19    padded_img = padarray(double(image), [half_k_row, half_k_col], 0, 'both');
20
21    % Initialize the output image
22    output = zeros(img_rows, img_cols);
23
24    % Iterate over each pixel (i, j) of the original image
25    for i = 1:img_rows
26        for j = 1:img_cols
27
28            % (i, j) in the output corresponds to the block centered
29            % at (i + half_k_row, j + half_k_col) in the padded image.
30
31            % Extract the local region from the padded image
32            % The region has the same size as the kernel
33            region = padded_img(i : i + kern_rows - 1, j : j + kern_cols - 1);
34
35            % Perform element-wise multiplication with the *flipped* kernel
36            % and sum the results (this is the dot product)
37            sum_val = sum(region .* flipped_kernel, 'all');
38
39            output(i, j) = sum_val;
40        end
41    end
42 end
```

Listing 1: Custom 2D convolution function (my_convolution.m).

```
1 function G_norm = my_gradient_norm(image, conv_function)
2     % Computes the 2-norm of the gradient image.
3     % It uses the custom convolution function passed as an argument.
4
5     % Define the Sobel kernels for X and Y gradients
6     Kx = [-1 0 1;
7           -2 0 2;
8           -1 0 1];
9
10    Ky = Kx'; % The Y-kernel is the transpose of the X-kernel
11
12    % Calculate Gx and Gy using the provided convolution function
13    Gx = conv_function(image, Kx);
14    Gy = conv_function(image, Ky);
15
16    % Compute the 2-norm (magnitude)
```

```

17 G_norm = sqrt(Gx.^2 + Gy.^2);
18 endrt(Gx.^2 + Gy.^2);
19 end

```

Listing 2: Gradient 2-Norm function (my_gradient_norm.m).

```

1 edge_map = edge_grad; % gradient norm obtained in part c
2 %edge_map = abs(edge_lap); % laplacian obtained in part c
3
4 % First, create a binary mask from the gradient image or laplacian
5 thresh_level = max(max(edge_map))*0.8; % This may need to be adjusted
6
7
8 % Crop the gradient map to throw away boundary pixels which have very high
9 % gradient values (due to padding with zeros from earlier steps)
10 cropped_edge_mask = edge_map(3:end-2, 3:end-2) > thresh_level;
11 % Bring the mask back to its old size
12 edge_mask = padarray(cropped_edge_mask, [2, 2], 0, 'both');
13
14 % Find the (row, col) coordinates of all edge pixels (edge mask created by
15 % thresholding)
16 [rows, cols] = find(edge_mask);
17
18 % Initial Guess: Centroid
19 center_x_guess = mean(cols);
20 center_y_guess = mean(rows);
21
22 % Improvement: Least-Squares Circle Fit
23 % We solve the linear system A*c = B for c = [c1, c2, c3]
24 % where x^2 + y^2 + c1*x + c2*y + c3 = 0
25 A = [cols, rows, ones(length(cols), 1)];
26 B = -(cols.^2 + rows.^2);
27 c = A \ B; % Solve using MATLAB's backslash operator
28
29 % Extract center (a, b) from solution
30 center_x_fit = -c(1) / 2;
31 center_y_fit = -c(2) / 2;
32 radius_fit = sqrt(center_x_fit^2 + center_y_fit^2 - c(3));
33
34 fprintf('Initial center guess: (x=%.1f, y=%.1f)\n', center_x_guess,
35         center_y_guess);
36 fprintf('Fitted center: (x=%.1f, y=%.1f)\n', center_x_fit, center_y_fit);
37 fprintf('Fitted radius: %.1f\n', radius_fit);
38
39 % --- Part (d-ii): Find Defects ---
40
41 % Convert all edge points to polar coordinates relative to the fitted center
42 dx = cols - center_x_fit;
43 dy = rows - center_y_fit;
44 theta = atan2(dy, dx);
45 rho = sqrt(dx.^2 + dy.^2); % This is the radius of each point
46
47 % Define a defect as any point whose radius is "far" from the fitted radius
48 tolerance = 5; % e.g., 2 pixels
49 is_defect = abs(rho - radius_fit) > tolerance;
50
51 defect_rows = rows(is_defect);
52 defect_cols = cols(is_defect);
53
54 % Show the final result
55 figure('Units', 'normalized', 'Position', [0.05 0.05 0.9 0.9]); % a big centered
56 figure

```

```

55 imshow(img_original, 'InitialMagnification', 'fit');           % fit image to
    window                                                     % preserve
56 axis image;
    aspect ratio
57 title('Defective Regions Identified','FontSize',24);
58 hold on;
59
60 % Plot the fitted circle (Capture handle)
61 t = linspace(0, 2*pi, 100);
62 h_circle = plot(center_x_fit + radius_fit*cos(t), center_y_fit + radius_fit*sin
    (t), 'b--', 'LineWidth', 2);
63
64 % Plot the defective points in red (Capture handle)
65 h_defects = plot(defect_cols, defect_rows, 'r.', 'MarkerSize', 12);
66
67 % --- ONE-LINE CODE: Plot the Central Point and capture its handle ---
68 % Plotted as a large magenta cross (m+). The handle is h_center.
69 h_center = plot(center_x_fit, center_y_fit, 'm+', 'MarkerSize', 16, 'LineWidth'
    , 3);
70
71 % Update the legend with the center point and its coordinates
72 legend([h_circle, h_defects, h_center], ...
73     {'Fitted Circle', 'Defective Points', sprintf('Center (%.1f, %.1f)',
    center_x_fit, center_y_fit)}, ...
74     'FontSize', 24, 'Location', 'NorthEast');
75
76 hold off;

```

Listing 3: Main analysis script for Parts (c) and (d).

B Supplementary Figures

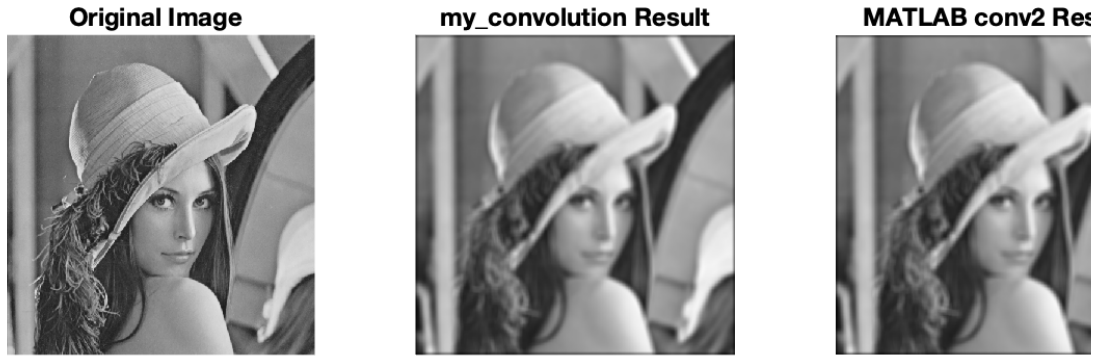


Figure 5: Validation of `my_convolution` (Part a). A test image (left) is convolved with a Gaussian kernel (size=11, sigma=7). The output of the custom function (middle) is visually identical to MATLAB's `conv2` (right).



Figure 6: Demonstration of `my_gradient_norm` (Part b). The original image (left) is processed to find the 2-norm of its gradient, highlighting all edges (right).

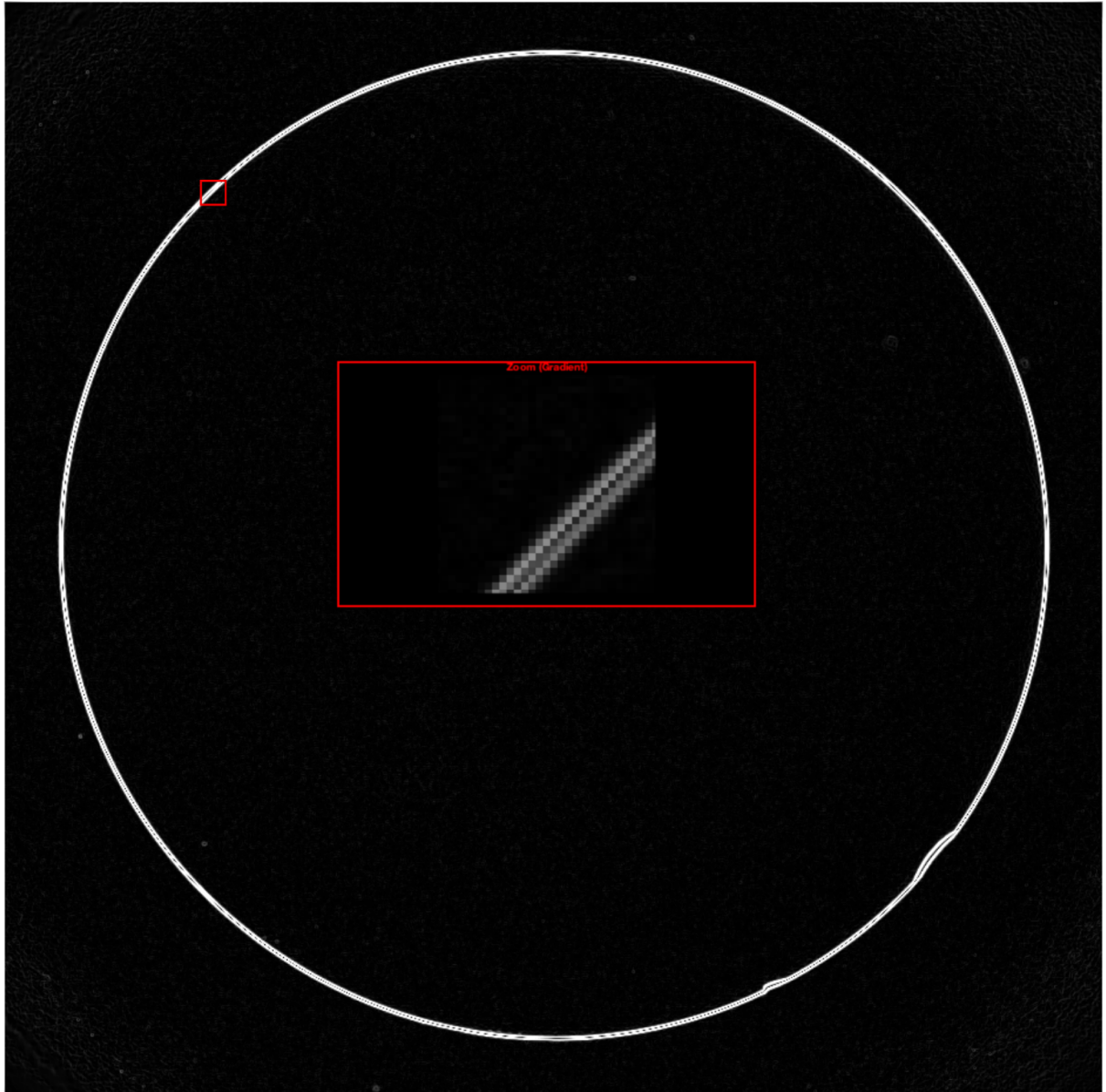


Figure 7: Large size version of gradient norm of the image in part C

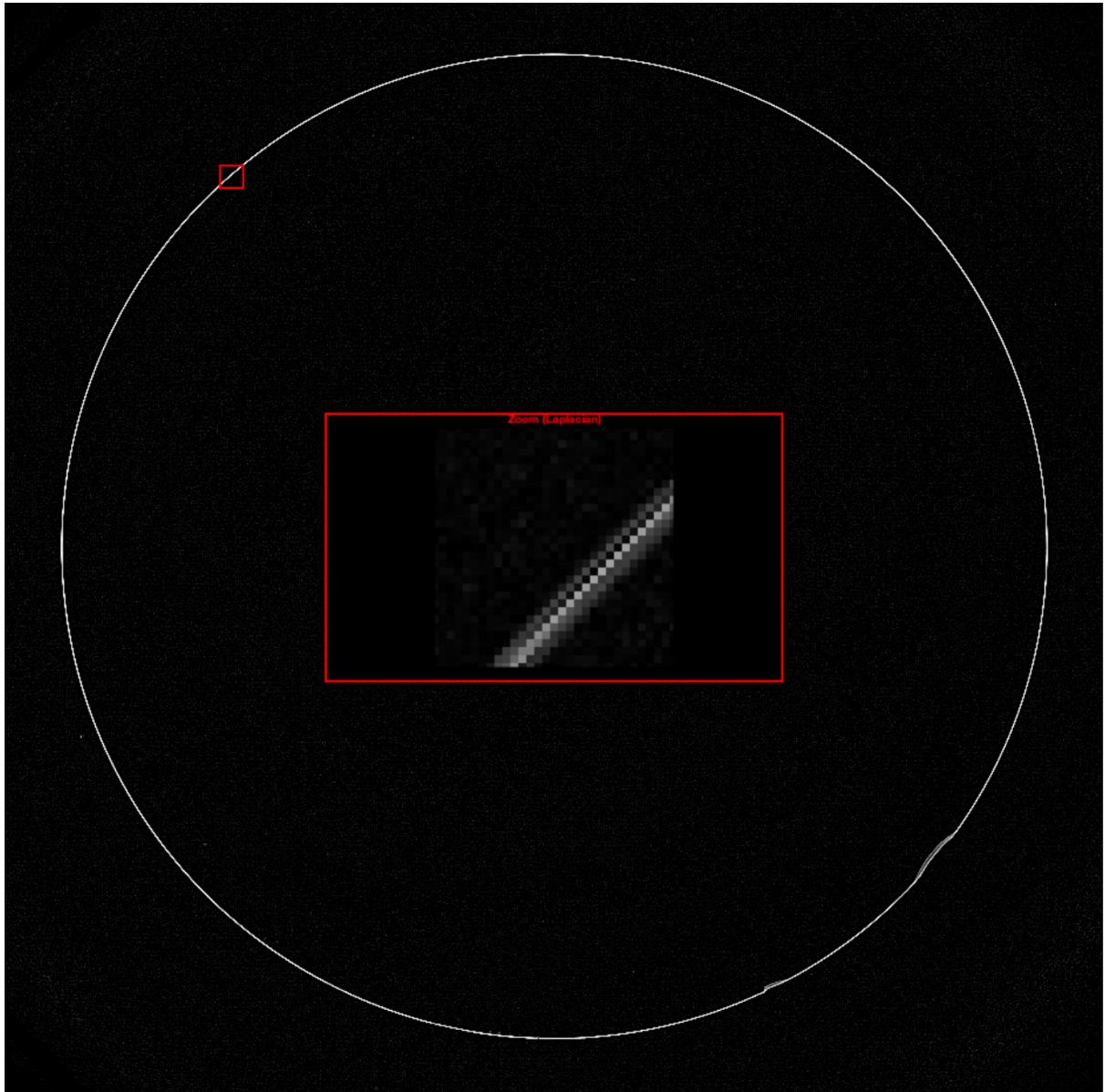


Figure 8: Large size version of Laplacian of the image in part C